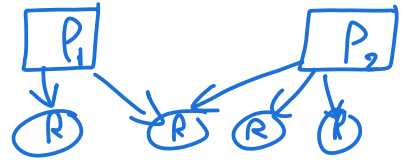


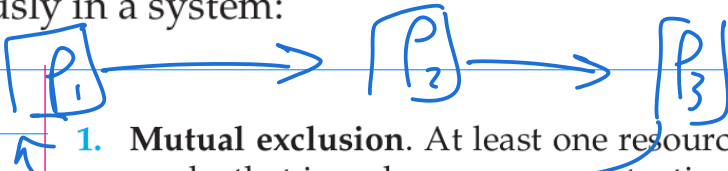
Deadlock



1. **Request.** The process requests the resource. If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.
2. **Use.** The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer).
3. **Release.** The process releases the resource.

7.2.1 Necessary Conditions

A deadlock situation can arise if the following four conditions hold simultaneously in a system:



1. **Mutual exclusion.** At least one resource must be held in a nonsharable mode; that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
2. **Hold and wait.** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
3. **No preemption.** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
4. **Circular wait.** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , ..., P_{n-1} is waiting for a resource held by P_n , and P_n is waiting for a resource held by P_0 .

7.2.2 Resource-Allocation Graph

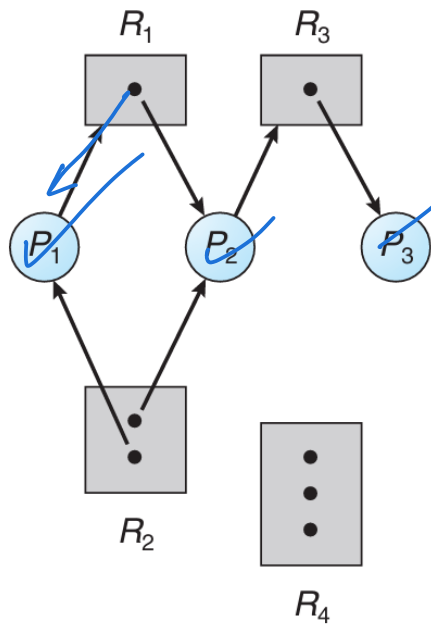
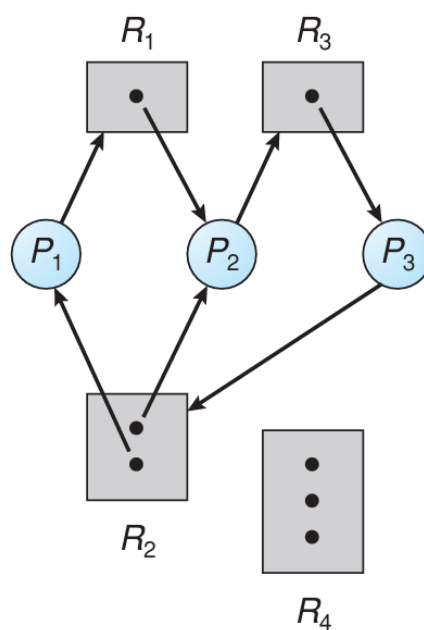


Figure 7.1 Resource-allocation graph.



— prevent
— detection
→ no dead lock

Figure 7.2 Resource-allocation graph with a deadlock.

7.5.3 Banker's Algorithm

Several data structures must be maintained to implement the banker's algorithm. These data structures encode the state of the resource-allocation system. We need the following data structures, where n is the number of processes in the system and m is the number of resource types:

R_1, R_2
3 5

• **Available.** A vector of length m indicates the number of available resources of each type. If $Available[j]$ equals k , then k instances of resource type R_j are available.

2 1

• **Max.** An $n \times m$ matrix defines the maximum demand of each process. If $Max[i][j]$ equals k , then process P_i may request at most k instances of resource type R_j .

1 0

• **Allocation.** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. If $Allocation[i][j]$ equals k , then process P_i is currently allocated k instances of resource type R_j .

1 1

• **Need.** An $n \times m$ matrix indicates the remaining resource need of each process. If $Need[i][j]$ equals k , then process P_i may need k more instances of resource type R_j to complete its task. Note that $Need[i][j]$ equals $Max[i][j] - Allocation[i][j]$.

7.5.3.1 Safety Algorithm

We can now present the algorithm for finding out whether or not a system is in a safe state. This algorithm can be described as follows:

1. Let **Work** and **Finish** be vectors of length m and n , respectively. Initialize **Work** = **Available** and **Finish** $[i]$ = **false** for $i = 0, 1, \dots, n - 1$.

2. Find an index i such that both

a. **Finish** $[i]$ == **false**

b. $Need_i \leq Work$

If no such i exists, go to step 4.

3. **Work** = **Work** + **Allocation** $_i$

Finish $[i]$ = **true**

Go to step 2.

4. If **Finish** $[i]$ == **true** for all i , then the system is in a safe state.

This algorithm may require an order of $m \times n^2$ operations to determine whether a state is safe.

7.5.3.2 Resource-Request Algorithm

Next, we describe the algorithm for determining whether requests can be safely granted.

Let $Request_i$ be the request vector for process P_i . If $Request_i[j] == k$, then process P_i wants k instances of resource type R_j . When a request for resources is made by process P_i , the following actions are taken:

1. If $Request_i \leq Need_i$, go to step 2. Otherwise, raise an error condition, since the process has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3. Otherwise, P_i must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to process P_i by modifying the state as follows:

$$\begin{aligned} Available &= Available - Request_i; \\ Allocation_i &= Allocation_i + Request_i; \\ Need_i &= Need_i - Request_i; \end{aligned}$$

If the resulting resource-allocation state is safe, the transaction is completed, and process P_i is allocated its resources. However, if the new state is unsafe, then P_i must wait for $Request_i$, and the old resource-allocation state is restored.

7.5.3.3 An Illustrative Example

To illustrate the use of the banker's algorithm, consider a system with five processes P_0 through P_4 and three resource types A , B , and C . Resource type A has ten instances, resource type B has five instances, and resource type C has seven instances. Suppose that, at time T_0 , the following snapshot of the system has been taken:

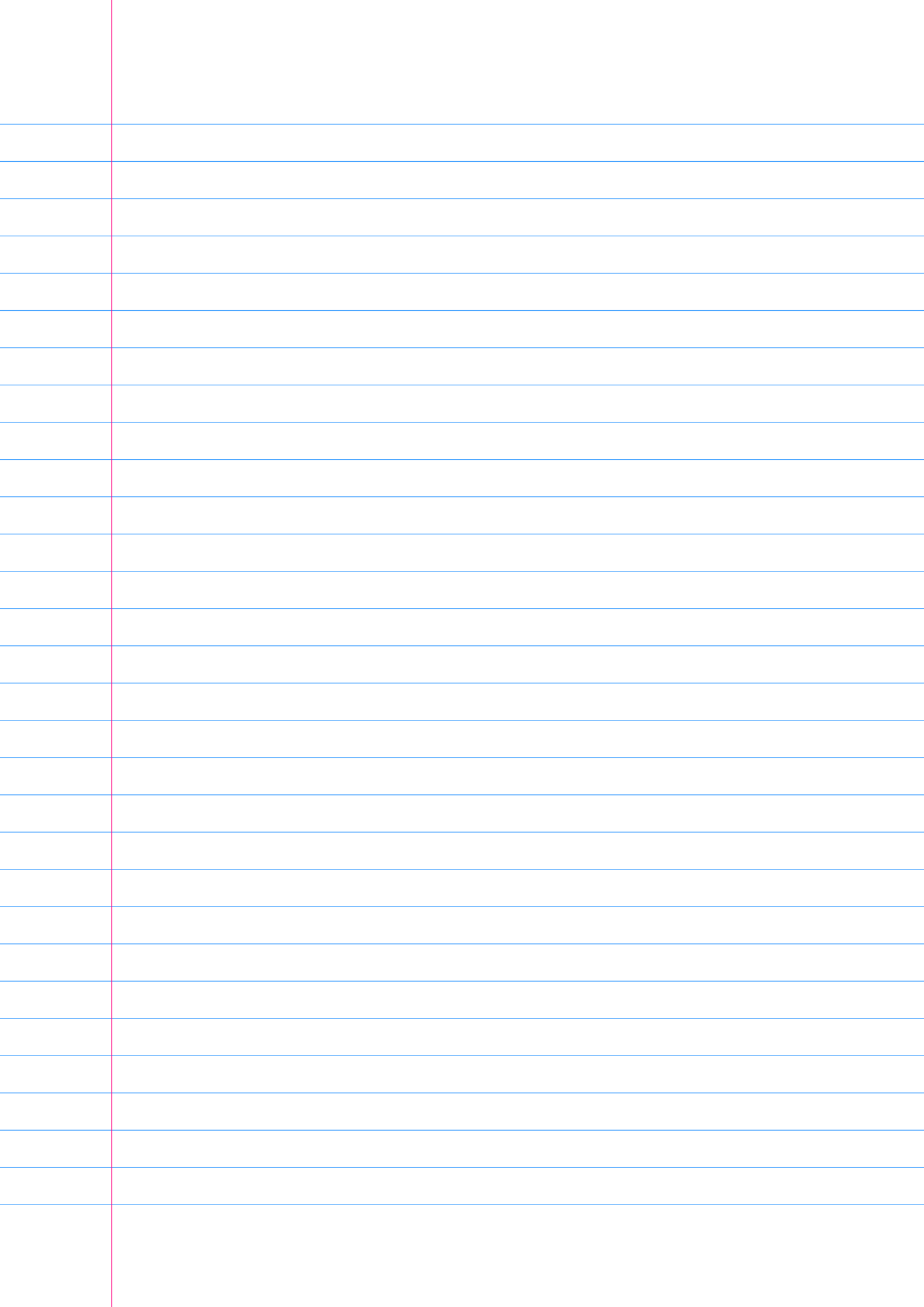
<u>Work</u>		<u>Allocation</u>	<u>Max</u>	<u>Available</u>	<u>Need</u>	
A B C		A B C	A B C	A B C	A B C	
3 3 2	P_0	0 1 0	7 5 3	3 3 2	7 4 3	P_0
	P_1	2 0 0	3 2 2		1 2 2	P_1
	P_2	3 0 2	9 0 2		6 0 0	P_2
	P_3	2 1 1	2 2 2		0 1 1	P_3
	P_4	0 0 2	4 3 3		4 3 1	P_4

The content of the matrix *Need* is defined to be $Max - Allocation$ and is as follows:

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>	<u>Need</u>	
	A B C	A B C	A B C	A B C	
P_0	0 1 0	7 5 3	3 3 2	7 4 3	1, 1, 2
P_1	2 0 0	3 2 2	5 3 2	1 2 2	(10, 5, 7)
P_2	3 0 2	9 0 2	7 4 3	6 0 0	
P_3	2 1 1	2 2 2	7 4 5	0 1 1	
P_4	0 0 2	4 3 3	7 5 5	4 3 1	
	2 2 2				

$P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1 \rightarrow P_2$

P_4 req (2, 2, 0)



Deadlock Detection

- **Available.** A vector of length m indicates the number of available resources of each type.
 - **Allocation.** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
 - **Request.** An $n \times m$ matrix indicates the current request of each process. If $Request[i][j]$ equals k , then process P_i is requesting k more instances of resource type R_j .
1. Let **Work** and **Finish** be vectors of length m and n , respectively. Initialize $Work = Available$. For $i = 0, 1, \dots, n-1$, if $Allocation_i \neq 0$, then $Finish[i] = false$. Otherwise, $Finish[i] = true$.
 2. Find an index i such that both
 - a. $Finish[i] == false$
 - b. $Request_i \leq Work$If no such i exists, go to step 4.
 3. $Work = Work + Allocation_i$
 $Finish[i] = true$
Go to step 2.
 4. If $Finish[i] == false$ for some $i, 0 \leq i < n$, then the system is in a deadlocked state. Moreover, if $Finish[i] == false$, then process P_i is deadlocked.

$P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$ work

no deadlock

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
$\times P_1$	2 0 0	2 0 2	0 1 0
$\times P_2$	3 0 3	0 0 1	
$\times P_3$	2 1 1	1 0 0	
$\times P_4$	0 0 2	0 0 2	

dead lock

$P_0 \rightarrow$

7.3 Consider the following snapshot of a system:

max- allocation

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C D	A B C D	A B C D
P_0	0 0 1 2	0 0 1 2	1 5 2 0
P_1	1 0 0 0	1 7 5 0	
P_2	1 3 5 4	2 3 5 6	
P_3	0 6 3 2	0 6 5 2	
P_4	0 0 1 4	0 6 5 6	

Answer the following questions using the banker's algorithm:

- What is the content of the matrix *Need*?
- Is the system in a safe state?
- If a request from process P_1 arrives for (0,4,2,0), can the request be granted immediately?

work

Available

Need

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>	<u>Need</u>
	A B C D	A B C D	A B C D	A B C D
P_0	0 0 1 2	0 0 1 2	1 5 2 0	0 0 0 0
P_1	1 0 0 0	1 7 5 0	1 5 3 2	0 7 5 0
P_2	1 3 5 4	2 3 5 6	2 8 8 6	1 0 0 2
P_3	0 6 3 2	0 6 5 2	2 14 11 8	0 0 2 0
P_4	0 0 1 4	0 6 5 6	2 14 12 12	0 6 4 2
			2 14 12 13	

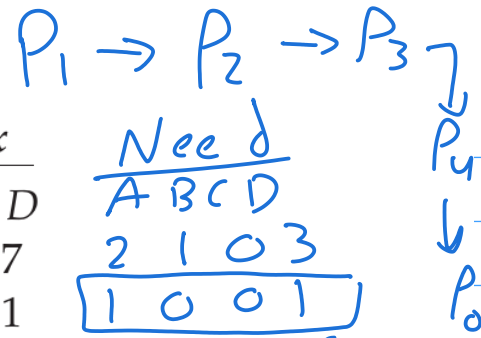
$P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

Yes

	<u>Allocation</u>	<u>Max</u>	<u>Work Available</u>	<u>Need</u>
	A B C D	A B C D	A B C D	A B C D
P₀	0 0 1 2	0 0 1 2	1 5 2 0	0 0 0 0
P ₁	1 0 0 0	1 7 5 0	1 1 0 0	0 3 3 0
P₂	1 3 5 4	2 3 5 6	1 1 1 2	1 0 0 2
P₃	0 6 3 2	0 6 5 2	2 4 6 6	0 0 2 0
P₄	0 0 1 4	0 6 5 6	2 10 9 8	0 6 4 2
			2 10 10 12	

P₀ → P₂ → P₃ → P₄ → P₁

7.22 Consider the following snapshot of a system:



Work

A	B	C	D
1	0	0	2
3	2	1	2
6	3	3	3
6	8	4	3
10	10	5	5

	<u>Allocation</u>	<u>Max</u>
	A B C D	A B C D
P_0	3 0 1 4	5 1 1 7
P_1	2 2 1 0	3 2 1 1
P_2	3 1 2 1	3 3 2 1
P_3	0 5 1 0	4 6 1 2
$\rightarrow P_4$	4 2 1 2	6 3 2 5

Need

A	B	C	D
2	1	0	3
1	0	0	1
0	2	0	0
4	1	0	?
2	1	1	3

Using the banker's algorithm, determine whether or not each of the following states is unsafe. If the state is safe, illustrate the order in which the processes may complete. Otherwise, illustrate why the state is unsafe.

a. *Available* = (0, 3, 0, 1)

b. *Available* = (1, 0, 0, 2)

7.23 Consider the following snapshot of a system:

<u>Need</u>		<u>Allocation</u>	<u>Max</u>	<u>Available</u> <i>work</i>
A B C D		A B C D	A B C D	A B C D
2 2 1 1	P_0	2 0 0 1	4 2 1 2	3 3 2 1
2 1 3 1	P_1	3 1 2 1	5 2 5 2	3 3 0 1
0 2 1 3	P_2	2 1 0 3	2 3 1 6	
0 1 1 2	P_3	1 3 1 2	1 4 2 4	
2 2 1 3	P_4	1 4 2 2	3 6 6 5	
		5		

Answer the following questions using the banker's algorithm:

- Illustrate that the system is in a safe state by demonstrating an order in which the processes may complete.
- If a request from process P_1 arrives for (1, 1, 0, 0), can the request be granted immediately?
- (c) If a request from process P_4 arrives for (0, 0, 2, 0), can the request be granted immediately?

dead lock